NAME : BISMA FATIMA

CLASS : CS A

ROLL NO: F24605036

COURSE : OBJECT ORIENTED PROGRAMMING

COURSE CODE : CS-123

LECTURER : MISS TAYYABA KALSOOM

POST LAB TASK WEEK 10

Task 1: Method Overloading – Calculator Operations

```
import java.util.Scanner;
// CLASS CALCULATOT
public class calculator{
    //WE ARE DOING COMPILE TIME POLYMORPHISM
    public int add(int a,int b){
        return a+b;
    }
    public double add(double a,double b){
        return a+b;
    }
    public int add(int a,int b,int c){
        return a+b+c;
    }
    //MAIN METHODE
    public static void main(String[] args){
        calculator obj=new calculator();   //OBJECT OF CLASS
        Scanner sc=new Scanner(System.in);
        //ASK USER TO ENTER VALUE OF INTEGERS
        System.out.println("enter value of 'a' integer:");
        int a=sc.nextInt();
        System.out.println("enter value of 'b' integer:");
        int b=sc.nextInt();
        //PRINT SUM OF TWO INTEGERS BY CALLING MRTHOD
        System.out.println("sum of both a and b integers:"+obj.add(a,b));
        //ASK USER TO ENTER VALUE OF DOUBLES
        System.out.println("\nenter value of 'A' Double:");
        double A=sc.nextDouble();
        System.out.println("enter value of 'B' Double:");
        double B=sc.nextDouble();
        //PRINT SUM OF DOUBLE BY CALLING METHOD
        System.out.println("sum of both A and B Doubles:"+obj.add(A,B));
        //ASK USER TO ENTER VALUE OF 3RD INTEGER
         System.out.println("\nenter value of 'c' integer:");
        int c=sc.nextInt();
        //PRINT SUM OF THREE INTEGER BY CALLING METHOD
        System.out.println("sum of both a,b,and c integers:"+obj.add(a,b,c));
    }
}
```

OUTPUT

```
enter value of 'a' integer:
3
enter value of 'b' integer:
3
sum of both a and b integers:6

enter value of 'A' Double:
2.0
enter value of 'B' Double:
2.0
sum of both A and B Doubles:4.0

enter value of 'c' integer:
3
sum of both a,b,and c integers:9
```

Task 2: Method Overloading – Printer Utility

```java
import java.util.Scanner;
public class Printer {
    // METHOD TO PRINT A STRING MESSAGE
    public void print(String message) {
        System.out.println(message);
    }
    // METHOD TO PRINT AN INTEGER NUMBER
    public void print(int number) {
        System.out.println(number);
    }
    // METHOD TO PRINT A MESSAGE A SPECIFIED NUMBER OF TIMES
    public void print(String message, int count) {
        for (int i = 0; i < count; i++) {
            System.out.println(message);
        }
    }
    //MAIN METHOD
    public static void main(String[] args) {
        // CREATING A SCANNER OBJECT TO TAKE INPUT FROM THE USER
        Scanner scanner = new Scanner(System.in);
        Printer obj = new Printer();
        // ASKING THE USER TO ENTER A STRING
        System.out.println("Enter a string:");
        String Message = scanner.nextLine();
        // ASKING THE USER TO ENTER AN INTEGER
        System.out.println("Enter an integer:");
```

```
        int Number = scanner.nextInt();
        // USING THE OVERLOADED METHODS
        obj.print(Message);          // PRINT THE STRING
        obj.print(Number);           // PRINT THE INTEGER
        obj.print(Message, Number); // PRINT THE STRING userNumber TIMES
    }
}
```

OUTPUT

```
Enter a string:
bisma fatima is here to write !!!
Enter an integer:
2
bisma fatima is here to write !!!
2
bisma fatima is here to write !!!
bisma fatima is here to write !!!
```

Task 3: Runtime Polymorphism – Appliance Control

```
// PARENT CLASS
class Appliance {
    // METHOD THAT CAN BE OVERRIDDEN BY CHILD CLASSES
    public void turnOn() {
        System.out.println("The appliance is turning on.");
    }
}
// CHILD CLASS FAN
class Fan extends Appliance {
    @Override
    public void turnOn() {
        System.out.println("The fan is spinning.");
    }
}
// CHILD CLASS  LIGHT
class Light extends Appliance {
    @Override
    public void turnOn() {
        System.out.println("The light is shining.");
    }
}
// CHILD CLASS  HEATER
class Heater extends Appliance {
```

```
    @Override
    public void turnOn() {
        System.out.println("The heater is warming up.");
    }
}
// MAIN CLASS
public class Main {
    //MAIN METHOD
    public static void main(String[] args) {
        // CREATING OBJECTS FOR EACH CHILD CLASS
        Appliance fan = new Fan();
        Appliance light = new Light();
        Appliance heater = new Heater();

        // CALLING OVERRIDDEN METHODS USING EACH OBJECT
        fan.turnOn();     // CALLS Fan's turnOn()
        light.turnOn();   // CALLS Light's turnOn()
        heater.turnOn();  // CALLS Heater's turnOn()
    }
}
```

OUTPUT

```
The fan is spinning.
The light is shining.
The heater is warming up.
```

Task 4: Upcasting – Employee Roles

```
// BASE CLASS
class Employee {
    // METHOD TO BE OVERRIDDEN BY SUBCLASSES
    public void work() {
        System.out.println("Employee is working.");
    }
}
// SUBCLASS  MANAGER
class Manager extends Employee {
    @Override
    public void work() {
```

```
        System.out.println("Manager is overseeing the team.");
    }
}
// SUBCLASS DEVELOPER
class Developer extends Employee {
    @Override
    public void work() {
        System.out.println("Developer is writing code.");
    }
}
// MAIN CLASS TO TEST UPCASTING AND DYNAMIC METHOD DISPATCH
public class Mainemployee {
    public static void main(String[] args) {
        // UPCASTING
        Employee manager = new Manager();
        manager.work(); // CALLS Manager's work() METHOD

        // UPCASTING
        Employee developer = new Developer();
        developer.work(); // CALLS Developer's work() METHOD
    }
}
```

OUTPUT

```
Manager is overseeing the team.
Developer is writing code.
```

Task 5: Runtime Polymorphism – Payment Gateways

```
import java.util.Scanner;
// BASE CLASS
class PaymentGateway {
    // METHOD TO BE OVERRIDDEN BY SUBCLASSES
    public void processPayment() {
        System.out.println("Processing payment using the default gateway.");
    }
}
// SUBCLASS  PAYPAL
class PayPal extends PaymentGateway {

    @Override
    public void processPayment() {
        Scanner scanner = new Scanner(System.in);
```

```
        System.out.println("Processing payment via PayPal...");
        System.out.print("Enter your PayPal email: ");
        String email = scanner.nextLine();
        System.out.println("Payment processed successfully for email: " + email);
    }
}
// SUBCLASS STRIPE
class Stripe extends PaymentGateway {

    @Override
    public void processPayment() {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Processing payment via Stripe...");
        System.out.print("Enter your Stripe card number: ");
        String cardNumber = scanner.nextLine();
        System.out.println("Payment processed successfully for card number  " + cardNumber);
    }
 }
// SUBCLASS JAZZCASH
class JazzCash extends PaymentGateway {

    @Override
    public void processPayment() {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Processing payment via JazzCash...");
        System.out.print("Enter your JazzCash mobile number: ");
        String mobileNumber = scanner.nextLine();
        System.out.println("Payment processed successfully for mobile number: " +
mobileNumber);
    }
}
// MAIN CLASS
public class Mainpaymeent {
    //MAIN METHOD
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Choose a payment gateway: PayPal, Stripe, or JazzCash");
        String choice = scanner.nextLine();
        PaymentGateway gateway;
        // USING BASE CLASS REFERENCE TO CALL THE METHOD FOR CHOSEN
GATEWAY
        switch (choice.toLowerCase()) {
            case "paypal":
                gateway = new PayPal();
                break;
```

```
            case "stripe":
                gateway = new Stripe();
                break;
            case "jazzcash":
                gateway = new JazzCash();
                break;
            default:
                System.out.println("Invalid choice. Please choose PayPal, Stripe, or JazzCash.");
                return;
        }
        // CALLING THE OVERRIDDEN METHOD
        gateway.processPayment();
    }
}
```

OUTPUT

```
Choose a payment gateway: PayPal, Stripe, or JazzCash
jazzcash
Processing payment via JazzCash...
Enter your JazzCash mobile number: 03339882304
Payment processed successfully for mobile number: 03339882304
```

Task 6: Area Calculator – Using Inheritance

```
// BASE CLASS
abstract class Shape {
    // ABSTRACT METHOD TO BE OVERRIDDEN BY SUBCLASSES
    public abstract double calculateArea();
}
// SUBCLASS SQUARE
class Square extends Shape {
    private double side;

    public Square(double side) {
        this.side = side;
    }

    @Override
    public double calculateArea() {
        return side * side;
    }
}
// SUBCLASS PARALLELOGRAM
```

```
class Parallelogram extends Shape {
    private double base;
    private double height;

    public Parallelogram(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return base * height;
    }
}
// SUBCLASS ELLIPSE
class Ellipse extends Shape {
    private double a;
    private double b;

    public Ellipse(double a, double b) {
        this.a = a;
        this.b = b;
    }

    @Override
    public double calculateArea() {
        return Math.PI * a * b;
    }
}
// MAIN CLASS
public class Mainarea {
    public static void main(String[] args) {
        // CREATE AN ARRAY OF SHAPE REFERENCES
        Shape[] shapes = {
            new Square(4),           // SIDE = 4
            new Parallelogram(5, 3),  // BASE = 5, HEIGHT = 3
            new Ellipse(2, 1)        // a = 2, b = 1
        };
        // LOOP FOR ARRAY
        for (int i = 0; i < shapes.length; i++) {
            System.out.println("The area of shape " + (i + 1) + " is: " + shapes[i].calculateArea());
        }
    }
}
```

OUTPUT

```
The area of shape 1 is: 16.0
The area of shape 2 is: 15.0
The area of shape 3 is: 6.283185307179586
```